Java Level 11 — DSA & Problem Solving

Part 1 — Core Patterns

Status

Level 11: In Progress

Covered in this part:

- ✅ Big-O Complexity
- ✅ Arrays & Hashing
- ✅ Two Pointers
- ✅ Sliding Window
- ✅ Prefix Sum
- ✅ Stack
- ✅ Monotonic Stack
- ✅ Queue / Deque Basics
- 🟡 Monotonic Deque / Sliding Window Maximum — Revisit

---

1. Big-O Complexity

Big-O describes how an algorithm's resource usage grows as the input size increases.

For interviews, focus mainly on:

- Time Complexity
- Space Complexity

Common Complexities

Complexity	Typical Example
`O(1)`	HashMap lookup
`O(log n)`	Binary Search
`O(n)`	Single traversal
`O(n log n)`	Efficient sorting
`O(n²)`	Nested loops
`O(2^n)`	Some recursive/backtracking solutions

Ignore constants:

```
O(2n)      → O(n)
O(5n + 10) → O(n)
O(n² + n)  → O(n²)
```

Interview Habit

Always be ready to explain:

```
Time Complexity
Space Complexity
```

---

2. Arrays + Hashing

HashMap and HashSet are commonly used to replace repeated searching with approximately `O(1)` lookup.

Recognition Cues

```
Need fast lookup?
→ HashMap / HashSet

"Have I seen X?"
→ HashSet

"Where did I see X?"
→ HashMap<Value, Index>

"How many X?"
→ HashMap<Value, Count>
```

---

3. Two Sum — HashMap Pattern

Problem

Given:

```
nums = [2,7,11,15]
target = 9
```

Return indices:

```
[0,1]
```

because:

```
2 + 7 = 9
```

Key Idea

For every number:

```
complement = target - current
```

Ask:

```
Have I already seen complement?
```

Implementation

```java
public static int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> map = new HashMap<>();

    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];

        if (map.containsKey(complement)) {
            return new int[]{map.get(complement), i};
        }

        map.put(nums[i], i);
    }

    return new int[]{-1, -1};
}
```

Complexity:

```
Time  → O(n)
Space → O(n)
```

Important

Check before inserting the current value.

This correctly handles:

```
nums = [3,3]
target = 6
```

---

4. Valid Anagram — Frequency Map

Problem

```
"anagram"
"nagaram"

→ true
```

Pattern

Count characters from the first string.

Then decrement/remove them while processing the second.

```
public static boolean isAnagram(String s, String t) {
    if (s.length() != t.length()) {
        return false;
    }

    Map<Character, Integer> map = new HashMap<>();

    for (char c : s.toCharArray()) {
        map.put(c, map.getOrDefault(c, 0) + 1);
    }
```

```
    for (char c : t.toCharArray()) {
        if (!map.containsKey(c)) {
            return false;
        }

        if (map.get(c) > 1) {
            map.put(c, map.get(c) - 1);
        } else {
            map.remove(c);
        }
    }

    return true;
}
```

Complexity:

```
Time  → O(n)
Space → O(k)
```

where `k` is the number of distinct characters.

---

5. Two Pointers

Two Pointers uses two indices that move according to some condition.

Common form:

```
left →            ← right
```

Recognition Cues

Think Two Pointers when you see:

- Sorted array
- Pair with target
- Palindrome
- Compare opposite ends
- Reverse
- Remove duplicates
- Optimization involving two boundaries

6. Valid Palindrome

```java
public static boolean isPalindrome(String s) {
    int left = 0;
    int right = s.length() - 1;

    while (left < right) {
        if (s.charAt(left) != s.charAt(right)) {
            return false;
        }

        left++;
        right--;
    }

    return true;
}
```

Complexity:

```
Time  → O(n)
Space → O(1)
```

---

7. Two Sum II — Sorted Array

Because the array is sorted:

```
sum < target
→ need bigger number
→ left++

sum > target
→ need smaller number
→ right--
```

Implementation

```java
public static int[] twoSumSorted(int[] nums, int target) {
    int left = 0;
    int right = nums.length - 1;

    while (left < right) {
        int sum = nums[left] + nums[right];

        if (sum == target) {
            return new int[]{left, right};
        }

        if (sum < target) {
            left++;
        } else {
            right--;
        }
    }

    return new int[]{-1, -1};
}
```

Complexity:

```
Time  → O(n)
Space → O(1)
```

---

8. Container With Most Water

Area:

```
width  = right - left
height = min(height[left], height[right])

area = width × height
```

Critical Rule

Always move the **shorter wall**.

Why?

The shorter wall limits the current area.

Moving the taller wall decreases width without giving us a chance to increase the limiting height.

```
public static int maxArea(int[] height) {
    int left = 0;
    int right = height.length - 1;

    int maxArea = 0;

    while (left < right) {
        int width = right - left;

        int h = Math.min(
                height[left],
                height[right]
        );

        maxArea = Math.max(
                maxArea,
                width * h
        );

        if (height[left] < height[right]) {
            left++;
        } else {
            right--;
        }
    }

    return maxArea;
}
```

Complexity:

```
Time  → O(n)
Space → O(1)
```

---

9. Sliding Window

Sliding Window is useful for **contiguous** sections of arrays or strings.

Recognition:

```
Array/String
+
Subarray/Substring/Contiguous
+
Condition/calculation
```

Then ask:

> Can I reuse information from the previous window instead of recalculating everything?

There are two major types:

```
Fixed Sliding Window
Variable Sliding Window
```

---

10. Fixed Sliding Window

Given:

```
nums = [2,1,5,1,3,2]
k = 3
```

Windows:

```
[2,1,5] → 8
[1,5,1] → 7
[5,1,3] → 9
[1,3,2] → 6
```

Instead of recalculating every window:

```
newSum =
oldSum
- leaving element
+ entering element
```

Maximum Sum Subarray of Size K

```java
public static int maxSumSubarray(int[] nums, int k) {
    int windowSum = 0;

    for (int i = 0; i < k; i++) {
        windowSum += nums[i];
    }

    int maxSum = windowSum;

    for (int end = k; end < nums.length; end++) {
        windowSum += nums[end];
        windowSum -= nums[end - k];

        maxSum = Math.max(maxSum, windowSum);
    }

    return maxSum;
}
```

Remember:

```
Build first window

Then repeatedly:

+ entering
- leaving

Update answer
```

Complexity:

```
Time  → O(n)
Space → O(1)
```

---

11. Maximum Average Subarray

Same fixed-window pattern.

```
public static double maxAverage(int[] nums, int k) {
    int currentWindowSum = 0;

    for (int i = 0; i < k; i++) {
        currentWindowSum += nums[i];
    }

    int maxWindowSum = currentWindowSum;

    for (int end = k; end < nums.length; end++) {
        currentWindowSum += nums[end];
        currentWindowSum -= nums[end - k];

        maxWindowSum = Math.max(
                maxWindowSum,
                currentWindowSum
        );
    }

    return (double) maxWindowSum / k;
}
```

---

12. Variable Sliding Window

Variable window does not have a predetermined size.

Mental model:

```
EXPAND RIGHT
     ↓
Check condition
     ↓
SHRINK LEFT when necessary
     ↓
Update answer
     ↓
Continue
```

General template:

```
int left = 0;

for (int right = 0; right < n; right++) {

    // Add right to window

    while (windowNeedsShrinking()) {
        // Remove left
        left++;
    }

    // Update answer
}
```

---

13. Longest Substring Without Repeating Characters

Problem:

```
"abcabcbb"

→ "abc"
→ 3
```

Use:

```
Sliding Window + HashSet
```

The HashSet represents characters currently inside the window.

```
public static int lengthOfLongestSubstring(String s) {
    Set<Character> window = new HashSet<>();

    int left = 0;
    int maxLength = 0;

    for (int right = 0; right < s.length(); right++) {
```

```
        while (window.contains(s.charAt(right))) {
            window.remove(s.charAt(left));
            left++;
        }

        window.add(s.charAt(right));

        maxLength = Math.max(
                maxLength,
                right - left + 1
        );
    }

    return maxLength;
}
```

Important Pitfall

When shrinking:

```
window.remove(s.charAt(left));
```

NOT:

```
window.remove(s.charAt(right));
```

The left pointer represents the element actually leaving the window.

Why `while`?

One new character may require removing multiple characters before the window becomes valid again.

---

14. Minimum Size Subarray Sum

Given positive integers:

```
nums = [2,3,1,2,4,3]
target = 7
```

Smallest qualifying subarray:

```
[4,3]

length = 2
```

Pattern

Expand using `right`.

Once:

```
sum >= target
```

the window is valid.

Because we're looking for the **minimum**, keep shrinking while it remains valid.

```
public static int minSubArrayLen(int target, int[] nums) {
    int left = 0;
    int sum = 0;

    int minLength = Integer.MAX_VALUE;

    for (int right = 0; right < nums.length; right++) {
        sum += nums[right];

        while (sum >= target) {
            minLength = Math.min(
                    minLength,
                    right - left + 1
            );

            sum -= nums[left];
            left++;
        }
    }

    return minLength == Integer.MAX_VALUE
            ? 0
            : minLength;
}
```

Important Distinction

Longest valid window

```
Expand

Invalid?
→ shrink until valid

Update MAX
```

Smallest valid window

```
Expand

Valid?
→ update MIN
→ shrink while still valid
```

This distinction is very important.

Why positive integers?

Adding on the right only increases the sum.

Removing from the left only decreases it.

Negative numbers destroy this monotonic behavior, so this simple sliding-window approach does not generally work.

---

15. Prefix Sum

Prefix Sum stores cumulative sums so range sums can be calculated efficiently.

Given:

```
nums = [2,4,1,7,3]
```

Prefer an extra leading zero:

```
prefix = [0,2,6,7,14,17]
```

Build:

```
public static int[] buildPrefixSum(int[] nums) {
    int[] prefix = new int[nums.length + 1];

    for (int i = 0; i < nums.length; i++) {
        prefix[i + 1] =
                prefix[i] + nums[i];
    }

    return prefix;
}
```

Then:

```
sum(left...right)

= prefix[right + 1] - prefix[left]
```

Example:

```
left = 1
right = 3

14 - 2 = 12
```

Complexity

Build:

```
O(n)
```

Each range query:

```
O(1)
```

If there are many queries, build the prefix array once rather than rebuilding it for every query.

---

16. Prefix Sum + HashMap

Very important interview combination.

Problem — Subarray Sum Equals K

```
nums = [1,2,3]
k = 3

answer = 2
```

Subarrays:

```
[1,2]
[3]
```

Core Equation

Suppose:

```
currentPrefix - previousPrefix = k
```

Rearrange:

```
previousPrefix = currentPrefix - k
```

Therefore, at every position ask:

```
Have I previously seen:

currentPrefix - k ?
```

Use:

```
HashMap<PrefixSum, Frequency>
```

Implementation

```
public static int subarraySum(int[] nums, int k) {
    Map<Integer, Integer> prefixCount =
            new HashMap<>();

    prefixCount.put(0, 1);

    int prefixSum = 0;
    int count = 0;

    for (int num : nums) {
        prefixSum += num;

        count += prefixCount.getOrDefault(
                prefixSum - k,
                0
        );

        prefixCount.put(
                prefixSum,
                prefixCount.getOrDefault(
                        prefixSum,
                        0
                ) + 1
        );
    }

    return count;
}
```

Complexity:

```
Time  → O(n)
Space → O(n)
```

Why `{0 → 1}` initially?

It represents the empty prefix.

This allows us to detect subarrays starting from index `0`.

For example:

```
prefixSum = k

prefixSum - k = 0
```

Important Pitfall

Do NOT simply:

```
count++;
```

when a previous prefix exists.

The same prefix sum may have appeared multiple times.

Instead:

```
count += prefixCount.get(prefixSum - k);
```

Every previous occurrence creates another valid subarray.

---

17. Stack

Stack follows:

```
LIFO

Last In
First Out
```

In Java prefer:

```
Deque<Integer> stack =
        new ArrayDeque<>();
```

Operations:

```
stack.push(x);
stack.pop();
```

```
stack.peek();
stack.isEmpty();
```

Think Stack when dealing with:

- Matching parentheses
- Nested structures
- Undo/history
- Expression evaluation
- Most recent unresolved item
- Next/previous greater/smaller

---

18. Valid Parentheses

Problem:

```
"{[]}" → true
"([)]" → false
```

Opening brackets go onto the stack.

Closing brackets must match the most recent opening bracket.

```
public static boolean isValid(String s) {
    Deque<Character> stack =
            new ArrayDeque<>();

    for (char c : s.toCharArray()) {

        if (c == ')' || c == ']' || c == '}') {

            if (stack.isEmpty()) {
                return false;
            }

            char open = stack.pop();

            if ((c == ')' && open != '(') ||
                (c == ']' && open != '[') ||
                (c == '}' && open != '{')) {

                return false;
            }
```

```
        } else {
            stack.push(c);
        }
    }

    return stack.isEmpty();
}
```

Complexity:

```
Time  → O(n)
Space → O(n)
```

Important Edge Case

Always check:

```
stack.isEmpty()
```

before `pop()`.

Otherwise:

```
"]"
```

throws an exception.

---

19. Monotonic Stack

A Monotonic Stack maintains values in an intentional increasing or decreasing order.

Used for:

- Next Greater Element
- Next Smaller Element
- Previous Greater Element
- Previous Smaller Element
- Daily Temperatures
- Stock Span
- Histogram problems

Mental model:

```
Stack = elements/indices still waiting for an answer
```

---

20. Next Greater Element

Given:

```
nums = [2,1,4,3]
```

Result:

```
[4,4,-1,-1]
```

Meaning:

```
2 → 4
1 → 4
4 → none
3 → none
```

Store **indices**, not values.

```java
public static int[] nextGreaterElement(int[] nums) {
    int[] answer = new int[nums.length];

    Arrays.fill(answer, -1);

    Deque<Integer> stack =
            new ArrayDeque<>();

    for (int i = 0; i < nums.length; i++) {

        while (!stack.isEmpty()
                && nums[i] > nums[stack.peek()]) {

            int previousIndex = stack.pop();

            answer[previousIndex] = nums[i];
```

```
        }

        stack.push(i);
    }

    return answer;
}
```

Mental model:

```
Current element arrives

Is it greater than something waiting
on the stack?

YES
→ current element is its answer
→ pop it

Repeat

Then current index starts waiting
```

Why O(n)?

Although there's a `while` inside the `for`:

```
each index pushed once
each index popped at most once
```

Therefore:

```
O(n)
```

not `O(n²)`.

---

21. Next Smaller Element

Almost identical.

Only change:

```
current > stackTop
```

to:

```
current < stackTop
```

```
public static int[] nextSmallerElement(int[] nums) {
    int[] answer = new int[nums.length];

    Arrays.fill(answer, -1);

    Deque<Integer> stack =
            new ArrayDeque<>();

    for (int i = 0; i < nums.length; i++) {

        while (!stack.isEmpty()
                && nums[i] < nums[stack.peek()]) {

            int previousIndex = stack.pop();

            answer[previousIndex] = nums[i];
        }

        stack.push(i);
    }

    return answer;
}
```

Complexity:

```
Time  → O(n)
Space → O(n)
```

---

22. Queue

Queue follows:

```
FIFO

First In
First Out
```

Java:

```
Queue<Integer> queue =
        new ArrayDeque<>();
```

Operations:

```
queue.offer(10);
queue.poll();
queue.peek();
```

Queues become particularly important later for:

```
BFS
Level-order traversal
Graph traversal
Task processing
```

---

23. Deque

Deque means:

```
Double Ended Queue
```

We can operate from both ends.

```
Deque<Integer> deque =
        new ArrayDeque<>();
```

Operations:

```
deque.offerFirst(x);
deque.offerLast(x);

deque.pollFirst();
deque.pollLast();

deque.peekFirst();
deque.peekLast();
```

---

24. Monotonic Deque — Sliding Window Maximum

Status: 🟡 Revisit

This pattern should be revisited during revision.

Problem:

```
nums = [1,3,-1,-3,5,3,6,7]
k = 3
```

Windows:

```
[1,3,-1]   → 3
[3,-1,-3]  → 3
[-1,-3,5]  → 5
[-3,5,3]   → 5
[5,3,6]    → 6
[3,6,7]    → 7
```

Result:

```
[3,3,5,5,6,7]
```

Core Idea

Deque stores **indices**.

Corresponding values remain in:

```
decreasing order
```

Therefore:

```
nums[deque.peekFirst()]
```

is always the maximum.

Front vs Back

This distinction is the key:

```
FRONT
→ current maximum
→ remove expired indices

BACK
→ remove smaller useless values
→ add current index
```

Why remove smaller values?

Suppose:

```
1 is waiting
```

and later:

```
3 arrives
```

`1` can never become maximum while `3` remains in the same/future relevant window because `3` is:

```
larger
AND
newer
```

So `1` is useless and can be discarded.

Implementation

```
public static int[] maxSlidingWindow(
        int[] nums,
        int k
) {
    Deque<Integer> deque =
            new ArrayDeque<>();

    int[] result =
            new int[nums.length - k + 1];

    for (int i = 0; i < nums.length; i++) {

        // Remove expired index
        if (!deque.isEmpty()
                && deque.peekFirst() <= i - k) {

            deque.pollFirst();
        }

        // Maintain decreasing values
        while (!deque.isEmpty()
                && nums[i] > nums[deque.peekLast()]) {

            deque.pollLast();
        }

        // Store current index
        deque.offerLast(i);

        // Record maximum once window is complete
        if (i >= k - 1) {
            result[i - k + 1] =
                    nums[deque.peekFirst()];
        }
    }

    return result;
}
```

Complexity:

```
Time  → O(n)
Space → O(k)
```

Revision Mental Model

Don't try to memorize the implementation initially.

Remember:

```
For each new element:

1. FRONT
   Remove expired index

2. BACK
   Remove smaller values

3. BACK
   Add current index

4. FRONT
   Read maximum
```

This topic should be traced manually again before interviews.

---

25. Pattern Recognition Cheat Sheet

HashMap / HashSet

```
"Have I seen X?"
→ HashSet

"Where did I see X?"
→ HashMap<Value, Index>

"How many X?"
→ HashMap<Value, Frequency>
```

Two Pointers

```
Sorted array
Pair target
Palindrome
Compare ends
Boundary optimization
```

Think:

```
left ↔ right
```

Sliding Window

```
Contiguous
Substring
Subarray
Longest/shortest
Fixed window size
```

Think:

```
right expands
left shrinks
```

Prefix Sum

```
Range sum
Cumulative sum
Repeated subarray sums
```

Think:

```
prefix[right + 1] - prefix[left]
```

Prefix Sum + HashMap

```
Number of subarrays
whose sum == k
```

Think:

```
previousPrefix
=
currentPrefix - k
```

Stack

```
Nested structures
Matching
Most recent unresolved item
```

Think:

```
LIFO
```

Monotonic Stack

```
Next greater
Next smaller
Previous greater
Previous smaller
```

Think:

```
"Indices waiting for an answer"
```

Queue

```
FIFO
BFS
Level-by-level processing
```

Monotonic Deque

```
Sliding window maximum/minimum
```

Think:

```
Front = answer

Back = maintain monotonic order
```

---

26. Common Mistakes From Practice

These are worth revising because they represent common interview bugs.

Minimum calculation

Wrong:

```
int min = Integer.MIN_VALUE;
min = Math.min(min, value);
```

Correct:

```
int min = Integer.MAX_VALUE;
```

Variable Sliding Window

When shrinking:

```
remove(nums[left]);
left++;
```

Do not remove the current `right` element unless the algorithm specifically requires it.

Prefix Sum

This representation:

```
prefix size = n + 1
```

is usually cleaner because it eliminates the `left == 0` special case.

Prefix Sum + HashMap

Wrong:

```
if (map.containsKey(target)) {
    count++;
}
```

Correct:

```
count += map.getOrDefault(target, 0);
```

Frequency matters.

Stack

Never blindly:

```
stack.pop();
```

when the stack may be empty.

Check:

```
stack.isEmpty()
```

first.

Monotonic Stack / Deque

Usually store:

```
indices
```

rather than values.

Indices let you:

- update the correct answer position
- determine whether an element expired
- still access its value through `nums[index]`

---

27. Complexity Cheat Sheet

Pattern	Typical Time	Typical Space
Hashing	`O(n)`	`O(n)`
Two Pointers	`O(n)`	`O(1)`
Fixed Sliding Window	`O(n)`	`O(1)`
Variable Sliding Window	`O(n)`	Depends
Prefix Sum Build	`O(n)`	`O(n)`
Prefix Range Query	`O(1)`	—
Prefix + HashMap	`O(n)`	`O(n)`
Stack	`O(n)`	`O(n)`
Monotonic Stack	`O(n)`	`O(n)`
Monotonic Deque	`O(n)`	`O(k)`

---

28. Revision Checklist

Before moving on, you should eventually be able to recognize these without being told the pattern.

Arrays / Hashing

- [x] Two Sum
- [x] Valid Anagram

Two Pointers

- [x] Valid Palindrome
- [x] Two Sum II
- [x] Container With Most Water

Sliding Window

- [x] Maximum Sum Subarray of Size K
- [x] Maximum Average Subarray
- [x] Longest Substring Without Repeating Characters
- [x] Minimum Size Subarray Sum

Prefix Sum

- [x] Range Sum
- [x] Subarray Sum Equals K

Stack

- [x] Valid Parentheses
- [x] Next Greater Element
- [x] Next Smaller Element

Queue / Deque

- [x] Queue fundamentals
- [x] Deque fundamentals
- [ ] Sliding Window Maximum — revisit

---

29. Level 11 Progress

```
✅ Big-O

✅ Arrays & Hashing
```

```
✅ Two Pointers

✅ Sliding Window

✅ Prefix Sum

✅ Stack

✅ Monotonic Stack

✅ Queue / Deque Basics

🟡 Monotonic Deque — revisit

⬜ Binary Search

⬜ Linked List

⬜ Trees / BST

⬜ Heap / Priority Queue

⬜ Graphs

⬜ BFS / DFS

⬜ Topological Sort

⬜ Union Find

⬜ Recursion

⬜ Backtracking

⬜ Greedy

⬜ Dynamic Programming

⬜ Intervals
```

Final Mental Model

Don't memorize individual solutions.

Train yourself to map problem language to patterns:

```
Need lookup?
→ Hashing

Sorted + pair/boundaries?
→ Two Pointers

Contiguous + longest/shortest?
→ Sliding Window

Repeated range/subarray sums?
→ Prefix Sum

Most recent unresolved item?
→ Stack

Next greater/smaller?
→ Monotonic Stack

FIFO / level-by-level?
→ Queue

Window maximum/minimum?
→ Monotonic Deque
```

That pattern recognition is the main goal of Level 11.